

Detailed Step-by-Step Trace with Pointer Illustrations

Given array:

Copy

```
[12, -7, -5, 70, -3, 15]
```

We'll walk through **every pointer move** and **swap**, showing ASCII snapshots at each micro-step.

1. Initial State

Copy

```
indices:  0   1   2   3   4   5
array:    [12, -7, -5, 70, -3, 15]
           L                       R
```

left = 0, right = 5

2. Iteration 1

2.1 Advance `left` to first non-negative

- Check `arr[0] = 12` → non-negative ⇒ stop

Copy

```
[12, -7, -5, 70, -3, 15]
  L
```

2.2 Retreat `right` to first negative

- Check `arr[5] = 15` ≥ 0 → `right--` → 4

Copy

```
[12, -7, -5, 70, -3, 15]
  L             R
```

- Check `arr[4] = -3 < 0` → stop

2.3 Swap and move pointers inward

- Swap `arr[0]` ↔ `arr[4]`
- New array: `[-3, -7, -5, 70, 12, 15]`
- Then `left++` → 1, `right--` → 3

Copy

```
indices:  0    1    2    3    4    5
array:    [-3, -7, -5, 70, 12, 15]
          L             R
```

3. Iteration 2

3.1 Advance `left` over negatives

- Check `arr[1] = -7 < 0` → `left++` → 2

Copy

```
[-3, -7, -5, 70, 12, 15]
  L       R
```

- Check `arr[2] = -5 < 0` → `left++` → 3

Copy

```
[-3, -7, -5, 70, 12, 15]
  L   R
```

- Check `arr[3] = 70 ≥ 0` → stop

3.2 Retreat `right` over non-negatives

- Check `arr[3] = 70` $\geq 0 \rightarrow$ `right--` $\rightarrow 2$

Copy

```
[-3, -7, -5, 70, 12, 15]
  R    L
```

3.3 Check for swap

- Now `left = 3`, `right = 2`
- Condition `left < right` fails $\rightarrow$ **no swap**, exit loop